Paper RAG 学生常见问题与解答（教师备案）

> 本文档整理学生在做这个项目时可能遇到的问题，以及对应的原因和解决办法，供授课前备案。 文中的报错信息大多是在本项目环境中**实际复现**得到的原文（2026 年 10 月，LangChain 1.x / Chroma 1.5.9）， 学生看到的报错应与此一致或相近。 配套文档：《项目讲解》（ `docs/项目讲解.md` ）。

排查问题的通用三步（建议先教给学生）：

1. **看终端**：后端的报错和建索引的进度都打印在运行 `python app.py` / `python ingest.py` 的终端里
2. **分段定位**：用 `/api/search` 单独测检索（不调用大模型），用 `python ingest.py --inspect` 单独看解析和切块
3. **看右侧面板**：回答不对时，先看检索到的片段里有没有答案。没有，说明是检索的问题；有，说明是生成的问题

```
curl -s localhost:8000/api/search -H 'Content-Type: application/json' \
     -d '{"query":"What datasets are used?","top_k":3}' | python -m json.tool
```

目录

- 一、环境与安装
- 二、API Key 与网络
- 三、数据下载
- 四、文档解析
- 五、文档切块
- 六、索引与向量库
- 七、检索
- 八、答案生成
- 九、网页与后端
- 十、检索评估
- 十一、部署到服务器
- 十二、概念与原理类问题
- 附：授课前的准备清单

一、环境与安装

Q1. 需要什么 Python 版本？用 3.9 报了一个看不懂的错

需要 **Python 3.10 及以上**（推荐 3.11 / 3.12）。代码中使用了 `dict | None` 这种类型写法，3.9 不支持，会在导入时报错：

```
TypeError: unsupported operand type(s) for |: 'type' and 'NoneType'
```

macOS 自带的 `/usr/bin/python3` 是 3.9，要另外安装新版本（python.org、Homebrew、conda 或 `uv` 都可以）。

Q2. `pip install` 很慢或者超时

国内访问 PyPI 慢，换镜像源：

```
pip install -r requirements.txt -i https://mirrors.aliyun.com/pypi/simple/
# 或永久设置
pip config set global.index-url https://mirrors.aliyun.com/pypi/simple/
```

清华源 `https://pypi.tuna.tsinghua.edu.cn/simple` 也可以。

Q3. 安装 pymupdf 时报包名不匹配

```
Failed to download and build `pymupdf==2.0`
Package metadata name `pymupdf-lite` does not match given name `pymupdf`
```

部分镜像上的 pymupdf 2.0 是一个元数据有问题的源码包。`requirements.txt` 里已经限制成 `pymupdf>=1.24,<2`。如果学生自己改了依赖，或者单独执行了 `pip install pymupdf`，让他们加上版本限制：`pip install "pymupdf<2"`。

Q4. 装好了包，运行时还是 `ModuleNotFoundError`

常见原因：

- **虚拟环境没激活**，包装到了别的 Python 里。用 `which python`（Windows 用 `where python`）确认当前用的是哪个 Python
- **装包和运行用的不是同一个 Python**。保险的做法是统一用 `python -m pip install ...`
- **报 `No module named 'rag'` 或 `'config'`**：要在**项目根目录**下运行 `python app.py` / `python ingest.py`。不要进到 `rag/` 目录里直接运行其中的文件，它们使用了相对导入，只能作为包被导入

Q5. 网上教程的 LangChain 代码，在这个项目的环境里报 ImportError

LangChain 1.x 把各种集成拆成了独立的包，很多老教程里的写法已经失效：

老写法（失效）	现在的写法
`from langchain.vectorstores import Chroma`	`from langchain_chroma import Chroma`
`from langchain.text_splitter import RecursiveCharacterTextSplitter`	`from langchain_text_splitters import RecursiveCharacterTextSplitter`
`from langchain.embeddings import OpenAIEmbeddings`	`from langchain_openai import OpenAIEmbeddings`
`from langchain.document_loaders import PyMuPDFLoader`	`from langchain_community.document_loaders import PyMuPDFLoader`
`from langchain.schema import Document`	`from langchain_core.documents import Document`
`LLMChain(...)`、`RetrievalQA.from_chain_type(...)`	LCEL 写法：`prompt | llm | StrOutputParser()`

建议学生查文档时以 LangChain 官方文档为准，并留意文档对应的版本。

Q6. Windows 上的命令和 macOS / Linux 不一样

操作	macOS / Linux	Windows（PowerShell）
激活虚拟环境	`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`
复制配置文件	`cp .env.example .env`	`copy .env.example .env`
多行命令续行	`\`	`` ` ``
curl 发 JSON	单引号包裹	建议直接在网页上测，或用 Python `requests`

PowerShell 提示「禁止运行脚本」时，执行 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`。

Q7. Windows 上安装 chromadb 失败，提示需要 Microsoft Visual C++

通常是因为 Python 版本太新或太旧，没有对应的预编译包，pip 只能从源码编译。解决办法：

1. 使用 **64 位** Python 3.10～3.12
2. 先 `python -m pip install --upgrade pip`，再重新安装
3. 仍然不行，再安装「Microsoft C++ Build Tools」

Q8. 运行时有一行 `The fitz API is deprecated` 警告

是 PyMuPDF 的提示，不影响运行，可以忽略。

二、API Key 与网络

Q9. 硅基流动返回 401

```
openai.AuthenticationError: Error code: 401 - {'code': 30014, 'data': None, 'message': 'Token
is invalid.'}
```

`SILICONFLOW_API_KEY` 不对。检查以下几点：

- 复制时是否多了空格、换行或引号（`.env` 里写成 `SILICONFLOW_API_KEY=sk-xxx` 即可，不要加引号）
- 是不是把 DeepSeek 的 Key 填到了硅基流动的位置
- Key 是否已在控制台被删除或禁用

Q10. DeepSeek 返回 401

```
Error code: 401 - {'error': {'message': 'Authentication Fails, Your api key: ****rong is
invalid ...', 'type': 'authentication_error', ...}}
```

`DEEPSEEK_API_KEY` 不对。排查方法同上。报错里的 `****rong` 是 Key 的最后几位，可以用来确认程序实际读到的是哪个 Key。

Q11. DeepSeek 报模型名不存在

```
Error code: 400 - {'error': {'message': 'The supported API model names are deepseek-flash,
deepseek-v4-pro, but you passed deepseek-4.1-flash. ...'}}
```

模型名要写 API 的 id，而不是产品名：DeepSeek-V4.1-Flash 对应的 id 是 **`deepseek-flash`**。网上旧教程里的 `deepseek-chat`、`deepseek-reasoner` 可能已经不可用，以报错里列出的名字为准。也可以直接查询可用的模型列表：

```
curl https://api.deepseek.com/models -H "Authorization: Bearer $DEEPSEEK_API_KEY"
```

Q12. 硅基流动报模型不存在

```
openai.BadRequestError: Error code: 400 - {'code': 20012, 'message': 'Model does not exist.
Please check it carefully.', 'data': None}
```

检查模型名是否完整，包括前缀，大小写也要一致：`Qwen/Qwen3-Embedding-4B`。

Q13. 报 402 或者余额不足

账户余额用完了，需要去对应平台的控制台充值，或者换一个 Key。建议提醒学生：

- semantic 切块会对每个句子调用一次 embedding，消耗比其他方法大得多
- 全量 1000 篇论文建一次索引，消耗大约是默认 200 篇的 5 倍

Q14. 报 429（请求过于频繁），或者全班一起建索引时特别慢

API 有频率限制（RPM / TPM）。**全班共用同一个 Key 时最容易遇到。** 解决办法：

1. **每个学生使用自己的 Key**（推荐）
2. **老师提前建好索引分发给学生**：只要 embedding 模型相同，`storage/` 目录可以直接复制给学生用，学生跳过建索引，直接问答
3. 课上建索引时用小子集：`--positives 20 --negatives 20`
4. 调小每次请求的批量：在 `.env` 里设 `EMBEDDING_BATCH_SIZE=16`

Q15. 改了 `.env` 不生效

两个常见原因：

1. **没有重启程序。** `.env` 只在启动时读取一次，改完要重启 `app.py`
2. **系统环境变量优先于 `.env`**：`load_dotenv` 默认不会覆盖已经存在的环境变量。例如终端里曾经执行过 `export PORT=8001`，`.env` 里的 `PORT` 就不起作用。用 `echo $PORT` 检查，`unset PORT` 清除

Q16. 偶尔报 `IncompleteRead`、`SSL: UNEXPECTED_EOF_WHILE_READING`、`Connection reset`

```
http.client.IncompleteRead: IncompleteRead(1597440 bytes read, 12450 more expected)
<urlopen error [SSL: UNEXPECTED_EOF_WHILE_READING] EOF occurred in violation of protocol
(_ssl.c:1010)>
```

网络不稳定，传到一半连接断了。项目里的下载脚本和评估脚本都带了自动重试，重新运行一般就能成功。频繁出现时：

- 如果开着代理（Clash 等），DeepSeek 和硅基流动都是国内服务，可以让它们**不走代理**（直连），或者换一个代理节点
- 校园网高峰期可以错峰，或者换手机热点试试

Q17. 开着代理，程序连不上 API，或者特别慢

Python 的 HTTP 库会读取 `HTTP_PROXY` / `HTTPS_PROXY` 环境变量。排查方法：

```
env | grep -i proxy          # 查看当前的代理设置
```

DeepSeek 和硅基流动直连就可以访问。需要直连时，可以在运行命令前临时去掉代理：`env -u HTTPS_PROXY -u HTTP_PROXY python app.py`。

三、数据下载

Q18. 下载论文很慢（200 篇要半个小时）

这是正常的：arXiv 要求脚本访问时每 3 秒最多 1 个请求，再加上传输时间，每篇大约 5～10 秒。**请不要让学生调小 `--delay`**，否则可能被 arXiv 暂时封禁 IP。建议：

- 作为课前作业，提前下载
- 课上用小子集：`--positives 20 --negatives 20`
- 老师提前下载好，用 U 盘或网盘分发 `papers/` 和 `data/` 目录

Q19. 下载到一半中断了

直接重新运行同一条命令。已经下载的文件会被跳过，只下载剩下的。PDF 先写入 `.part` 临时文件，写完才改名，所以不会留下损坏的 PDF。

Q20. 访问 Hugging Face 失败（下载元数据或标准段落时）

设置镜像后重新运行：

```
export HF_ENDPOINT=https://hf-mirror.com      # Windows: $env:HF_ENDPOINT="https://hf-mirror.com"
python scripts/download_open_ragbench.py
```

`download_open_ragbench.py` 和 `eval_retrieval.py` 都支持这个环境变量。

Q21. 每个学生下载到的论文是一样的吗?

一样。脚本使用了固定的随机种子（`--seed 42`），只要参数相同，选出的论文就相同。这样课上讨论同一个问题时，大家的结果可以对得上。

Q22. 这个数据集能用吗？有版权问题吗?

Open RAG Benchmark 的许可是 **CC BY-NC 4.0**：允许非商业使用（免费课程没问题），使用时要注明来源。论文 PDF 本身的版权属于各自的作者，建议**让学生运行脚本自己下载**，不要直接打包分发 PDF。

Q23. 想用自己的论文，可以吗?

可以。把 PDF 放进 `papers/`，然后重新建索引即可。也支持 `.md` 和 `.txt` 文件。但是评估脚本只能评估 Open RAG Benchmark 里的论文，因为只有它们有标准答案。

四、文档解析

Q24. 某篇论文解析出来几乎没有文字

很可能是**扫描版 PDF**（每一页都是图片），PyMuPDF 只能抽取文字层，抽不出图片里的字。表现是建索引日志里这篇论文的字符数很少。解决办法是先做 OCR，比如用 `ocrmypdf` 给 PDF 加上文字层，或者使用 PaddleOCR、MinerU 等工具。

Q25. 公式、表格变成了乱码或者错乱的数字

这是 PDF 文本抽取的固有局限：公式在 PDF 里是一个个独立摆放的符号，表格是一个个独立摆放的文本框。改进方向（可以作为进阶练习）：

- 使用专门针对论文的解析工具，例如 Marker、MinerU、Nougat、Docling，它们能输出 Markdown 和 LaTeX 格式的公式
- 用 `PyMuPDFLoader(..., extract_tables="markdown")` 抽取表格

Q26. 双栏论文的文字顺序是乱的

项目里已经设置了 `PyMuPDFLoader(..., sort=True)`，让文本按阅读顺序排列，大部分双栏论文都能正确处理。少数排版复杂的论文（比如图表穿插在两栏中间）仍可能有错乱，需要更强的版面分析工具（见 Q25）。

Q27. 右侧显示的论文标题不对

只有约 10% 的论文在 PDF 元数据里写了标题，其余的标题是根据「第一页字号最大的横排文字」猜出来的（见 `rag/parser.py` 的 `guess_title`）。这是一种启发式方法，少数特殊排版会猜错。标题只用于显示，不影响检索。

Q28. 参考文献的内容被检索出来了

检查 `.env` 里的 `SKIP_REFERENCES` 是不是 `true`（默认就是）。如果某篇论文的参考文献标题不是单独成行的 `References` / `Bibliography` / `参考文献`，比如写成了 `REFERENCES AND NOTES`，正则匹配不到，就不会被丢弃。可以修改 `rag/parser.py` 里的 `_REFERENCES_RE`。

Q29. 答案在附录里的问题，怎么都检索不到

这是项目中一个**有意保留的缺陷**：遇到 References 后，后面的内容全部丢弃，附录也一起丢了。评估结果显示，约 12% 的问题因此永远检索不到（「上限」只有 88.3%）。这是一个很好的进阶练习：让学生修改解析逻辑，只去掉参考文献、保留附录，再用评估脚本验证效果。

Q30. 建索引时提示「解析失败，已跳过 xxx.pdf」

这个 PDF 文件损坏，或者格式比较特殊。程序会跳过它，继续处理其他论文。可以尝试用浏览器打开这个 PDF 确认是否正常，必要时删掉后重新下载。

五、文档切块

Q31. chunk_size 是按字符算还是按 token 算?

本项目按**字符数**计算（Python 的 `len()`）。大致换算：

- 英文：800 字符 ≈ 130 个单词 ≈ 170～200 个 token
- 中文：800 字符 ≈ 500～800 个 token

所以同样的 chunk_size，中文 chunk 包含的信息量和 token 数，都和英文不一样。

Q32. chunk_size 和 overlap 应该设多少?

没有标准答案，要用评估结果说话。经验值：chunk_size 300～1000 字符，overlap 取 chunk_size 的 10%～20%。本项目实测（见《项目讲解》第 13.5 节）：

配置	Passage@5
recursive:400:60	79.2%
recursive:800:120	77.9%
recursive:1600:200	72.7%

小块检索更精准，但每个片段携带的上下文更少，回答质量不一定更好。

Q33. 设置 overlap 后报错

```
ValueError: Got a larger chunk overlap (300) than chunk size (200), should be smaller.
```

overlap 必须小于 chunk_size。

Q34. 有的 chunk 末尾挂着下一节的标题（比如以 `4 Experiments` 结尾）

这是 overlap 造成的：相邻块之间有重叠，前一块的末尾就带上了下一块的开头。这是正常现象。

Q35. 为什么没有很短的 chunk? 标题 `Abstract` 去哪了?

递归切分遇到「短标题 + 超长段落」时，会把标题单独切成一小块。项目里会把少于 50 个字符的碎片合并到下一块的开头，所以标题不会丢，只是和下一段正文合在了一起（见 `rag/chunker.py` 中 `pending` 的逻辑）。

Q36. semantic 切块非常慢，还很费钱

semantic 方法要对**每一个句子**调用 embedding API，计算相邻句子之间的语义距离，调用量是其他方法的好几倍。建议只在小子集上演示：

```
python scripts/download_open_ragbench.py --positives 10 --negatives 0
python ingest.py --inspect --method semantic
```

Q37. 中文论文能切好吗?

可以。递归切分的分隔符里包含了中文标点（`。` `！` `？` `；` `，`），semantic 切块的分句规则也兼顾了中文。

六、索引与向量库

Q38. 建索引要多久？终端好像卡住了

实测 200 篇论文约 1.9 万个 chunk，需要 **15～20 分钟**，速度取决于 API 响应和网络情况。终端会持续打印 `1920/19289` 这样的进度。只要数字在增长，就是在正常运行。

Q39. 换了 embedding 模型（或者改了维度）之后报错

```
chromadb.errors.InvalidArgumentError: Collection expecting embedding with dimension of 2560,
got 1024
```

或者网页提示：

```
索引由 BAAI/bge-m3 构建，当前模型是 Qwen/Qwen3-Embedding-4B，请重建索引
```

不同模型（或不同的 `EMBEDDING_DIMENSIONS`）生成的向量维度不同，也不在同一个向量空间里，旧索引不能继续用。**必须重建索引。** 注意：只改了 `EMBEDDING_DIMENSIONS`、没有换模型时，程序的模型名检查发现不了，会直接报上面的维度错误。

Q40. 用命令行重建索引后，网页报 `Collection ... does not exist`

```
NotFoundError: Error getting collection: Collection [e8c90f1d-...] does not exist.
```

`app.py` 在运行时，另一个终端里的 `python ingest.py` 重建了索引：旧的集合被删除、新建了一个，而 `app.py` 还拿着旧集合的引用。**重建完索引后，重启 `app.py` 即可。** 如果不想重启，就在网页的「设置」里点「重建索引」，它和网页服务是同一个进程，不会出这个问题。

Q41. 建索引中途断了（网络错误、手动中断），索引还能用吗?

不能放心使用。建索引一开始会**清空旧索引**，然后分批写入，中断时只写入了一部分。重新完整地跑一遍即可。

Q42. `storage/` 目录很大

Qwen3-Embedding-4B 默认输出 2560 维向量，1.9 万个 chunk 的向量库约 306MB。想减小体积，可以在 `.env` 里设置 `EMBEDDING_DIMENSIONS=1024`（或 512），然后重建索引。Qwen3-Embedding 支持截短维度，检索效果会有少量下降，这也是一个很好的对比实验。`storage/eval/` 是评估脚本的缓存，可以直接删除。

Q43. 怎么彻底清空索引?

停止程序后删除 `storage/` 目录，然后重新运行 `python ingest.py`。

Q44. 在网页上点了「重建索引」，很久都没有反应

建索引需要十几分钟，网页会一直显示「构建中」，进度打印在运行 `app.py` 的终端里。重复点击会提示「索引正在构建中，请稍候」（409）。论文较多时，推荐用命令行 `python ingest.py` 建索引。

Q45. 能不能不重新建索引，直接用别人建好的？

可以。只要 **embedding 模型和维度都相同**，把别人的 `storage/` 目录（`chroma/` 和 `index_info.json`）复制过来就能直接用。还要注意 Chroma 的版本最好一致，可以用 `deploy/requirements.lock` 里锁定的版本。

七、检索

Q46. ⚠️ 检索结果完全不相关，但程序没有任何报错

最隐蔽的一个坑，请重点提醒学生。如果自己写代码对接硅基流动时，没有给 `OpenAIEmbeddings` 设置 `check_embedding_ctx_length=False`，就会出现这个问题：

```
OpenAIEmbeddings(model=..., base_url="https://api.siliconflow.cn/v1", api_key=...,
                 check_embedding_ctx_length=False)   # ← 必须加
```

原因：默认值为 `True` 时，LangChain 会先用 OpenAI 的分词器把文本转成 **token id 数组**再发送。硅基流动不会报错，但返回的是错误的向量。我们实测过：同一句话两种方式得到的向量，余弦相似度只有约 0.3，不同句子之间的相似度也区分不开。改完以后**必须重建索引**。

其他可能导致检索不相关的原因：

- 换了模型却没有重建索引（见 Q39）
- 问题太笼统，比如「这篇论文讲了什么」：200 篇论文里，向量库不知道「这篇」指哪篇
- 答案在附录里（见 Q29），或者在图片、表格里（见 Q25）

Q47. 相关性分数为什么只有 0.5～0.7，从来不接近 1？

这是正常的。问题和论文片段在长度和表达方式上差别很大，余弦相似度不会很高。**分数的绝对值意义不大，排名才是关键。** 不同 embedding 模型的分数分布也不一样，不要用固定的阈值（比如「低于 0.8 就丢弃」）去套所有模型。

Q48. 用中文问英文论文，能检索到吗？

可以。Qwen3-Embedding 是多语言模型，中文问题和英文段落可以映射到同一个向量空间里。实测中用中文提问，能检索到英文论文中的相关片段。

Q49. Top-K 设多少合适?

一般 3～10。K 越大，命中答案的概率越高（Recall@10 > Recall@5），但送进大模型的资料也越长，成本越高，无关内容也越多，可能干扰回答。默认值是 5。

Q50. 右侧面板显示的「检索查询」和我输入的不一样

这是**多轮查询改写**：从第二轮开始，系统会让大模型结合对话历史，把问题改写成独立、完整的句子，比如把「它的分数是多少」改写成「RAG 论文在 NQ 数据集上的分数是多少」。如果改写结果不理想，可以在「设置」里关闭这个功能。

Q51. 为什么不用 LangChain 常见的 `vectorstore.as_retriever()`?

`as_retriever()` 返回的只有文档，**不带分数**，而右侧面板要展示相关性分数，所以用了 `similarity_search_with_relevance_scores`。注意 Chroma 里还有一个名字相近的方法：`similarity_search_by_vector_with_relevance_scores`。它的名字里虽然有 relevance，但返回的其实是**距离**（越小越相似），不是相关性分数。用之前要看清楚文档。

八、答案生成

Q52. 回答里出现了论文里没有的内容（幻觉）

先看右侧的检索结果：

- **检索结果里没有答案**：是检索的问题。按照 prompt 的规则，模型应该说明「在已有论文中没有找到相关信息」，补充的通用知识要标注「非论文内容」。实测中模型基本能遵守这一点
- **检索结果里有答案，模型却答错了**：是生成的问题。可以在 `rag/generator.py` 里加强 SYSTEM_PROMPT 的约束，比如「只能根据参考资料回答」

Q53. 回答里的引用编号和右侧的片段对不上

编号对应的是**这一轮**检索到的片段。点击历史中的某条回答，右侧会切换回那一轮的检索结果。如果还是对不上，说明模型引用错了，这是大模型偶尔会犯的错误。

Q54. 开了「深度思考」以后回答变慢了

正常现象。思考模式下，模型会先输出一段思考过程，再给出回答，总耗时更长，token 消耗也更多。思考过程可以在回答上方展开查看。

Q55. 聊的轮数越多，回答越慢

每次请求都会带上历史对话（默认最近 6 轮），轮数越多，输入越长。可以调小 `.env` 里的 `MAX_HISTORY_TURNS`，或者点「新对话」重新开始。

Q56. 回答用的是英文 / 中英混杂

SYSTEM_PROMPT 里规定了「用户用什么语言提问就用什么语言回答」。论文术语保留英文是有意为之。如果希望全部用中文，可以修改 `rag/generator.py` 里的规则 3。

九、网页与后端

Q57. 启动时报端口被占用

```
ERROR:    [Errno 48] error while attempting to bind on address ('127.0.0.1', 8000): address
already in use
```

（Linux 上是 `Errno 98`，Windows 上是 `Errno 10048`。）8000 端口已经被别的程序占用了，可能是另一个还在运行的 `app.py`。两种解决办法：

- 在 `.env` 里改端口：`PORT=8001`
- 找到并关闭占用端口的程序：macOS / Linux 用 `lsof -i :8000`，Windows 用 `netstat -ano | findstr 8000`

Q58. 直接双击 `static/index.html` 打开，页面不能用

前端需要通过后端接口获取数据，必须先运行 `python app.py`，再在浏览器里访问 `http://127.0.0.1:8000`，不能直接打开 HTML 文件。

Q59. 回答显示的是原始 Markdown 符号（`**`、`#`），没有渲染

前端从 CDN（cdn.jsdelivr.net）加载 Markdown 渲染库，网络访问不了 CDN 时，会自动退化为纯文本显示。这不影响功能。如果需要离线渲染，可以把 `marked.min.js` 和 `purify.min.js` 下载到 `static/` 目录，再修改 `index.html` 里的引用地址。

如果**其他格式都正常，只有个别加粗处露出了 `**`**，那是另一个原因：按照 CommonMark 规范，`**` 紧挨着中文全角标点时，比如 `**溢出效应（spillover）**`，不会被识别为加粗。项目里的 `static/app.js` 已经用 `fixCjkBold` 处理了这种情况。学生如果自己写前端，也会遇到这个问题。

Q60. 修改了前端代码，刷新页面没有变化

浏览器缓存了旧文件。强制刷新：Windows 按 `Ctrl+F5`，macOS 按 `Cmd+Shift+R`。

Q61. 页面顶部显示「尚未建立索引」

还没有建索引，或者 `storage/index_info.json` 不存在。运行 `python ingest.py`，或者在网页的「设置」里点「重建索引」。

Q62. 同学用我的地址访问不了

默认只监听 `127.0.0.1`，也就是只有本机能访问。想让同一局域网内的同学访问，在 `.env` 里设置 `HOST=0.0.0.0`，然后把你的局域网 IP 发给同学（例如 `http://192.168.1.23:8000`）。还要确认操作系统的防火墙允许这个端口。

Q63. 用中文输入法打字，按回车选词时把消息发出去了

前端已经处理了这种情况（输入法组字时按回车不会发送）。如果学生用的浏览器比较老仍有问题，可以按 `Shift+Enter` 换行，再点击「发送」按钮发送。

Q64. 回答不是一个字一个字出来的，而是等很久后一次性显示

响应被中间的代理缓冲了。常见于以下几种情况：

- 部署时在前面加了 Nginx，需要在配置里关闭缓冲：`proxy_buffering off;`
- 使用了云服务商的 HTTP 代理或 CDN，改用 TCP 端口映射即可（见 Q71）
- 公司或学校网络的代理会缓冲响应，换一个网络环境试试

十、检索评估

Q65. Recall@K 和 MRR 是什么意思?

- **Recall@K**：在所有问题里，正确答案出现在检索结果前 K 名中的比例。例如 Passage Recall@5 = 75.6%，表示 75.6% 的问题，含有答案的段落出现在了前 5 个检索结果中
- **MRR@10**（Mean Reciprocal Rank）：对每个问题，取第一个正确结果排名的倒数（第 1 名得 1 分，第 2 名得 0.5 分，第 3 名得 0.33 分……没进前 10 名得 0 分），再对所有问题取平均。它衡量的是正确结果排得有多靠前

Q66. 为什么 Paper@1 有 97%，Passage@1 却只有 51%?

找对**论文**很容易：这 200 篇论文的学科差异很大，问题里的专业词汇很容易对上正确的论文。但是找对**段落**难得多：同一篇论文里有几十个片段，主题都相近。真正决定模型能否答对的，是段落级的指标。

Q67. 「上限」是什么？为什么不是 100%?

上限是指：在索引的所有 chunk 中，**至少存在一个**能命中标准段落的 chunk 的问题比例，不管它排第几。它没有达到 100%，说明有些问题的标准段落根本没有进入索引，最主要的原因是附录被丢弃了（见 Q29）。Passage@K 不可能超过上限。

Q68. 命中是怎么判定的？会不会误判?

数据集的标准段落和我们的 chunk 来自不同的 PDF 解析器，文字不会逐字相同，所以评估脚本比较的是「词三元组」（连续 3 个单词）的重合度。满足任一条件即算命中：chunk 中 ⩾ 30% 的三元组来自标准段落，或

者 chunk 包含标准段落 ⩾ 50% 的三元组。验证结果显示，约 95% 的 800 字符 chunk 都能明确对应到某一个段落。

它是近似判定，会有少量误差：chunk 越大，重合比例越容易被稀释，所以大 chunk 的得分会略被低估。阈值定义在 `scripts/eval_retrieval.py` 顶部的 `CHUNK_SHARE` 和 `SECTION_SHARE`，可以调整。

Q69. 两次对比实验的结果差了几个百分点，能说明哪个方法更好吗?

不一定。默认的对比子集只有约 150 个问题，几个百分点的差异可能只是随机波动。要下结论，可以增加论文数（`--papers 50`），或者换几个随机种子（`--seed 1`、`--seed 2`）多跑几次，看结论是否稳定。

Q70. 能评估我自己的论文吗?

评估需要「问题 + 标准答案所在的位置」这样的标注数据。自己的论文没有这种标注，可以用大模型自动生成：让模型读每个段落，出几个问题，把这个段落记为标准答案。这种方法叫合成评估集，RAGAS 等工具都支持。

十一、部署到服务器

Q71. 服务器是共享 IP（NAT）的，怎么访问?

共享 IP 的服务器只有内网 IP，服务商把公网 IP 的部分端口转发给它。例如本课程使用的七牛云实例：公网 `10010` → 内网 `22`（SSH），公网 `10026` → 内网 `8000`（网站）。所以：

- SSH 要带上映射后的端口：`ssh -p 10010 root@<公网IP>`。直接连 22 端口，到达的是另一台机器，会报 `Permission denied (publickey)`
- 网站要在服务商的控制台添加「公网端口 → 内网 8000」的 **TCP 端口映射**，然后用 `http://<公网IP>:<公网端口>` 访问
- 不要用服务商的 HTTP 代理，它可能缓冲响应，导致流式输出失效（见 Q64）

Q72. 服务器上需要重新建索引吗?

不需要，也不建议。把本机建好的 `storage/` 目录上传到服务器，在服务器的 `.env` 里设置 `READ_ONLY=true` 即可。在只读模式下，网页会隐藏「重建索引」，接口也会拒绝重建请求。服务器上要安装和本机**完全相同版本**的依赖（`deploy/requirements.lock`），否则 Chroma 可能读不了索引。

Q73. 部署后外网访问不了

依次检查：

1. 服务是否在运行：`systemctl status paper-rag`
2. 在服务器上能否访问：`curl localhost:8000/api/status`
3. 监听地址：`.env` 里是否设置了 `HOST=0.0.0.0`（只监听 `127.0.0.1` 时，外部访问不了）
4. 防火墙：`ufw status`，以及云服务商的安全组规则

5. 端口映射：对于共享 IP 的服务器，是否映射了正确的内网端口（见 Q71）

Q74. 服务挂了怎么办？怎么看日志？

```
systemctl status paper-rag        # 查看运行状态
journalctl -u paper-rag -f        # 实时查看日志
systemctl restart paper-rag       # 重启服务
```

systemd 配置了 `Restart=always`，程序崩溃后会自动重启。

Q75. 网站公开后，API Key 会不会被盗刷？

Key 存在服务器上，不会发送到浏览器，所以访客看不到它。但任何能打开网站的人都可以提问，每次提问都会消耗你的额度。可以采取以下措施：

- 开启只读模式，防止访客触发重建索引（项目已支持）
- 给网站加访问密码
- 在 DeepSeek 和硅基流动的控制台，为部署专门创建一个 Key，并设置额度上限
- 不再需要时，及时关掉服务

Q76. 服务器需要多大内存？

实测服务运行时占用约 470MB 内存（200 篇论文，1.9 万个 2560 维向量）。2GB 内存的服务器就够了。论文更多时，内存占用随向量数量增长，可以通过降低向量维度（`EMBEDDING_DIMENSIONS`）来减小占用。

Q77. 浏览器提示网站「不安全」

因为用的是 http，不是 https。这不影响使用。如果需要 https，要准备一个域名和证书，再在前面加一层 Nginx 或 Caddy 反向代理。用 Nginx 时，记得关闭缓冲（`proxy_buffering off;`），否则流式输出会失效。

十二、概念与原理类问题

Q78. RAG 和微调（Fine-tuning）有什么区别？什么时候用哪个？

	RAG	微调
知识存放在哪	外部知识库，查询时检索	模型参数里
更新知识	加文档、重建索引，几分钟	重新训练，成本高
可追溯	能给出处（就是右侧面板）	不能
适合	事实性知识、经常变化的资料、需要引用来源	改变模型的说话风格、输出格式、特定任务能力

论文问答属于典型的「事实性知识 + 需要出处」，所以用 RAG。

Q79. DeepSeek 的上下文窗口有 100 万 token，为什么不把论文全文直接塞给模型?

1. **规模**：200 篇论文约 1100 万字符，约 280 万 token，已经远远超过了上下文窗口。真实场景中的知识库往往更大
2. **成本和速度**：每次提问都把全部论文送进模型，费用高，响应也慢
3. **效果**：上下文太长时，模型容易忽略中间部分的内容（「Lost in the Middle」现象）
4. **可追溯**：RAG 能明确告诉用户，答案依据的是哪一段原文

对于单篇或少数几篇论文，直接塞进上下文也是可行的方案，两者可以结合使用。

Q80. 为什么用余弦相似度，而不是欧氏距离?

余弦相似度只看两个向量的**方向**，不看长度，衡量的是语义上像不像。很多 embedding 模型输出的向量已经做过归一化（长度为 1），这时余弦相似度和欧氏距离给出的排序是一样的，但余弦相似度的取值范围固定（−1～1），更直观。Chroma 默认使用 L2 距离，所以项目里显式设置了 `{"hnsw": {"space": "cosine"}}`。

Q81. 向量数据库和普通数据库有什么区别?

普通数据库按**精确条件**查询（`WHERE title = 'xxx'`），向量数据库按**相似度**查询（找出和这个向量最接近的 K 个）。向量数量很大时，逐一比较太慢，向量数据库使用 HNSW 等近似最近邻索引，牺牲极少的精度，换来数量级的速度提升。

Q82. 为什么用 LangChain? 不用可以吗?

可以不用。RAG 的每一步都不复杂，用几个基础库（PyMuPDF、openai SDK、chromadb）也能实现。用 LangChain 的好处是：

- 各个组件都有统一的接口，换模型、换向量库只需要改一行
- 是业界最常用的框架，学生学到的东西可以迁移到工作中

代价是多了一层抽象，出问题时要了解它内部做了什么，比如 Q46 的 `check_embedding_ctx_length`、Q51 的分数和距离。

Q83. Embedding 模型怎么选?

参考 MTEB 排行榜（中文看 C-MTEB），再结合以下几点考虑：

- **语言**：是否需要跨语言检索
- **领域**：通用文本还是专业文献
- **向量维度**：影响存储和速度
- **部署方式**：调 API 还是本地运行

选好后，最好用自己的评估集（比如本项目的 `eval_retrieval.py`）实测对比，不要只看排行榜。

Q84. 不想调 API，能在本地运行 embedding 模型吗?

可以。把 `rag/embeddings.py` 换成 `langchain_huggingface` 的 `HuggingFaceEmbeddings`，加载 `BAAI/bge-m3` 等模型即可。需要注意：

- 需要安装 `sentence-transformers` 和 PyTorch，第一次运行会下载模型（bge-m3 约 2.3GB）。国内设置 `HF_ENDPOINT=https://hf-mirror.com`
- **Apple Silicon（M 系列）Mac 的坑**：如果用的是 x86_64 版本的 Python（比如在 Rosetta 下安装的旧版 Miniconda），只能装到 PyTorch 2.2，会报 `PyTorch >= 2.6 is required`。用 `python -c "import platform; print(platform.machine())"` 检查，输出应该是 `arm64`
- 没有 GPU 时，建索引会比调 API 慢

Q85. 能换成别的大模型吗? 比如 OpenAI、通义千问，或者本地的 Ollama?

可以。只需要修改 `rag/generator.py` 里创建 `self.llm` 的那一行：

```
from langchain_openai import ChatOpenAI
self.llm = ChatOpenAI(model="qwen-plus", api_key=...,
base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")

from langchain_ollama import ChatOllama      # 本地模型
self.llm = ChatOllama(model="qwen2.5:7b")
```

还要注意两点：

- `thinking` 这个 `extra_body` 参数是 DeepSeek 特有的，换成其他模型后要删掉
- 只有 DeepSeek 会把思考过程放在 `reasoning_content` 里，其他模型不会返回这部分内容

Q86. 检索效果还能怎么提升?

按投入产出比排序：

1. **修复解析问题**：保留附录、处理表格（「上限」提升了，检索效果才有提升空间）
2. **调整切块参数**：用评估脚本找到合适的 chunk_size
3. **重排序（Rerank）**：先召回 20 个候选，再用 Reranker 精排出 5 个。硅基流动提供 `Qwen/Qwen3-Reranker-4B`、`BAAI/bge-reranker-v2-m3`
4. **混合检索**：BM25 关键词检索 + 向量检索，对专有名词、数字类问题帮助大
5. **上下文增强**：给 chunk 加上「论文标题 + 章节名」前缀后再向量化
6. **查询扩展**：HyDE（先让模型写一个假设答案，再用它去检索）、多查询改写

附：授课前的准备清单

- □ **API Key**：决定让学生自己申请，还是由老师统一提供。统一提供时注意频率限制（Q14）和额度上限
- □ **预先建好索引**：老师提前跑好 200 篇论文的 `storage/`，分发给网络不好、或者来不及建索引的学生（Q45）

- □ **预先下载数据**：把 `papers/` 和 `data/` 打包备用，以防课上 arXiv 或 Hugging Face 访问不畅（Q18、Q20）
- □ **准备小子集**：课上做对比实验用 20+20 篇论文，避免每个配置都要等十几分钟（Q36、Q38）
- □ **确认 Python 版本**：通知学生提前安装 Python 3.10+，并配置好 pip 镜像（Q1、Q2）
- □ **准备示例问题**：从 `data/open_ragbench/eval_queries.json` 里挑几个问题，最好同时包含答案在正文、答案在附录、答案在表格里的例子，用于课堂演示
- □ **重点讲解的坑**：`check_embedding_ctx_length`（Q46）、换模型后要重建索引（Q39）、重建索引后要重启服务（Q40）